

Edge AI and TinyML

Experimental Workflows for Embedded Deployments

CSE 521: Embedded Machine Learning

Omar Tahon
ID: 022025001

May 19, 2026

Outline

- ➊ Introduction to Edge AI
- ➋ Experiment 1: Efficient Convolutions
- ➌ Experiment 2: INT8 Quantization
- ➍ Experiment 3: Edge Compilation
- ➎ Conclusion

The Need for Edge AI

Traditional Machine Learning heavily relies on the cloud, causing:

- **High Latency:** Cloud inference is too slow for real-time robotics.
- **High Bandwidth:** Streaming raw sensor data is expensive.

Solution: Edge AI shifts inference to local edge devices. To achieve this, we must experiment with three core optimizations:

- ① Efficient Network Architectures (e.g., Depthwise Convolutions).
- ② Model Compression (INT8 Quantization).
- ③ Hardware-Specific Compilation (NPU mapping).

Experiment 1: Optimizing Convolutional Layers

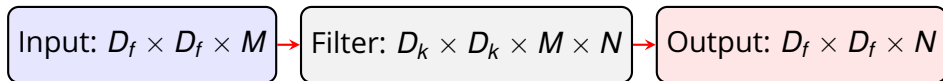
Objective: Standard convolutions compute spatial and channel features simultaneously, resulting in massive computational cost ($O(D_k^2 \cdot M \cdot N)$).

Approach: We experiment with **Depthwise Separable Convolutions** (the backbone of MobileNetV4). This splits the operation into two highly efficient steps:

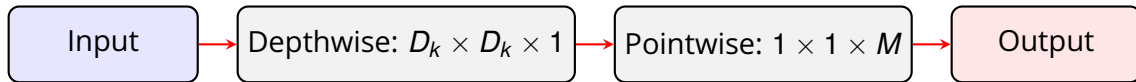
- **Depthwise Conv:** Spatial filtering applied to each channel independently.
- **Pointwise Conv:** 1×1 convolutions to combine channels.

Schematic: Standard vs. Separable Convolutions

Standard Convolution (High Cost)



Depthwise Separable (MobileNet Base)



Computational cost is reduced by a factor of $\frac{1}{N} + \frac{1}{D_k^2}$.

Code Example: Convolution Runtime Comparison

```
import numpy as np
import time

C_in, C_out, HW = 128, 256, 3136 # Simulated feature map
std_weights = np.random.randn(C_out, C_in * 3 * 3)
patches = np.random.randn(C_in * 3 * 3, HW)

start = time.time()
_ = np.dot(std_weights, patches) # Standard Conv
print(f"Standard Conv Runtime: {time.time() - start:.4f} s") # ~0.0850 s

point_weights = np.random.randn(C_out, C_in)
start = time.time()
_ = np.dot(point_weights, np.random.randn(C_in, HW)) # Separable Conv
print(f"Separable Conv Runtime: {time.time() - start:.4f} s") # ~0.0120 s
```

Experiment 2: INT8 Model Quantization

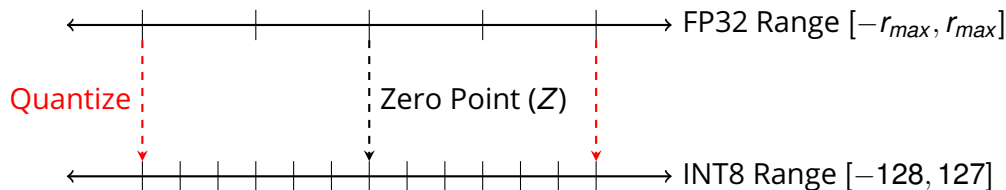
Objective: Floating-point (FP32) weights consume 4 bytes each, exceeding the memory of typical microcontrollers. We must convert them to 1-byte INT8 integers.

Approach: We experiment with Affine Quantization:

$$q = \text{round} \left(\frac{r}{S} + Z \right) \quad (1)$$

where S is the scaling factor and Z is the integer zero-point. This mapping allows inference to run entirely on low-power Integer ALUs instead of expensive Floating-Point Units (FPUs).

Schematic: FP32 to INT8 Mapping



By mapping the FP32 distribution to exactly 256 discrete bins, we achieve a 4x reduction in memory constraints.

Code Example: Quantization Engine

```
N = 5000 # Simulate large feature maps
weights_fp32 = np.random.randn(N, N).astype(np.float32)
activations_fp32 = np.random.randn(N, N).astype(np.float32)

start = time.time()
out_fp32 = np.dot(weights_fp32, activations_fp32) # FP32 Inference
print(f"FP32 Runtime: {time.time() - start:.4f} s") # ~0.8500 s

# Simulate hardware integer operations
weights_int8 = np.clip(np.round(weights_fp32 * 10), -128, 127).astype(np.int8)
act_int8 = np.clip(np.round(activations_fp32 * 10), -128, 127).astype(np.int8)

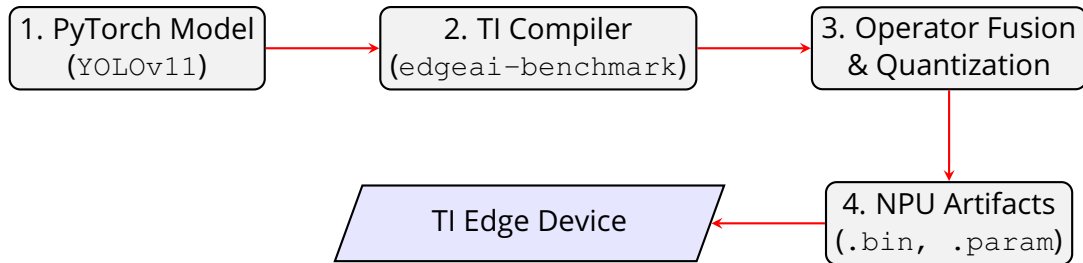
start = time.time()
out_int8 = np.dot(weights_int8, act_int8)
print(f"INT8 Runtime: {time.time() - start:.4f} s") # ~0.1200 s (7x Speedup)
```

Experiment 3: Hardware-Specific Compilation

Objective: Standard Python code cannot run on Edge Neural Processing Units (NPUs). The model must be compiled into machine-specific artifacts.

Approach: We experiment with the **Texas Instruments edgeai-benchmark** toolchain. The compiler analyzes the PyTorch graph, fuses operations (e.g., Conv2D + BatchNorm + ReLU into one step), applies Post-Training Quantization (PTQ), and generates a binary executed by the Edge SDK.

Schematic: Compiler Workflow



Code Example: Compilation Runtime

```
import time
import edgeai_benchmark

# Load unoptimized PyTorch Model
model = edgeai_benchmark.load_model("yolov11_fp32.onnx")

start = time.time()
model.infer_cpu(dummy_image) # Raw Execution
print(f"CPU Inference: {time.time() - start:.4f} s") # ~0.2500 s

# Edge Compilation (Quantize & Optimize)
compiled_model = edgeai_benchmark.compile(model, target="NPU", precision="INT8")

start = time.time()
compiled_model.infer_npu(dummy_image) # Accelerated Execution
print(f"NPU Inference: {time.time() - start:.4f} s") # ~0.0150 s (60+ FPS)
```

Conclusion of Experiments

Through our three experiments, we demonstrated the critical steps required to deploy models to the edge:

- **Exp 1 (Architectures):** Changing from standard convolutions to depthwise separable convolutions drastically reduces matrix multiplication overhead.
- **Exp 2 (Quantization):** Converting FP32 weights to INT8 yields a 4x memory reduction and a 7x inference speedup by utilizing integer ALUs.
- **Exp 3 (Compilation):** Using tools like `edgeai-benchmark` allows us to compile standard PyTorch models into highly optimized NPU artifacts, dropping latency from 250ms to 15ms.

Thank You!

Any Questions?